

# COMP3311

Week 5 Tutorial



# Agenda

✓ Week 5 notices

---

✓ Week 4 key concepts

---

✓ Tutorial questions

---

# Week 5 notices

- NO quiz this week :)
- Help sessions are still available (timetable on WebCMS)
- Assignment 1 due 23:59 Friday 3 July (Week 5 Friday) i.e. tmr
  - Whenever you submit, please check on a CSE computer first that your SQL code works on a fresh database (spec would also encourage this).
  - Manually check your code yourself, as well as run the autotests

# Week 4 Key Concepts

## Overview

- SQL functions
- PLpgSQL functions

# Week 4 Key Concepts

## Custom Types

- every table doubles as a type — e.g. if you **CREATE TABLE** called Accounts, you can use Accounts as a type itself
- custom tuple types can also be defined manually
- usable in custom aggregates

```
CREATE TYPE type_name AS (att1 TEXT, att2 INTEGER);  
  
DROP TYPE IF EXISTS type_name;
```



# Week 4 Key Concepts

## SQL functions

- like queries but accept arguments to vary output
- run a **SELECT** internally; return scalar, tuple, or table/setof
- no control flow (loops, conditionals) — use PLpgSQL for that

```
-- Function template, can have any return type
-- Common to use underscore in front of argument names to not be confused with table column names
CREATE OR REPLACE FUNCTION func_name(_arg1 TEXT, _arg2 INTEGER,...)
RETURNS INTEGER/TEXT/CUSTOM_TYPE/SETOF TYPE/TABLE(att1 type, att2 type) AS $$
    SELECT      t.col1, t.col2
    FROM        table t
    WHERE       t.x = _arg1 -- Argument name
               AND t.y = $2 -- 2nd argument, used when argument isn't given a name
$$ LANGUAGE SQL; -- Determines type of function
```

# Week 4 Key Concepts

## PL/pgSQL Functions

- same return types as SQL functions (scalar, tuple, setof)
- but full procedural control: variables, **IF** statements, **FOR** loops, storing query results in variables
- must explicitly **RETURN** a value
- verbose to write — semicolons everywhere, harder to debug

```
-- Function template, can have any return type
-- Common to use underscore in front of argument names to not be confused with table column names
CREATE OR REPLACE FUNCTION func_name(_arg1 TEXT, _arg2 INTEGER,...)
RETURNS INTEGER/TEXT/Custom_Type/SETOF Type/TABLE(att1 type, att2 type) AS $$
    DECLARE
        -- Can declare variables here
        var1 INTEGER = 0;
        var2 TEXT;
    BEGIN
        -- Function logic, please indent this like how you normally indent
        RETURN something;
    END; -- Semicolon after END is necessary
$$ LANGUAGE PLpgSQL; -- Determines type of function
```

# Week 4 Key Concepts

## PL/pgSQL variables and SELECT ... INTO

- in the **DECLARE** section of a PL/pgSQL function, you can declare variables to store values into
- you can use these variables to store specific values returned from a query by using **INTO** in the **SELECT** statement (very common pattern)

```
-- Function template, can have any return type
CREATE OR REPLACE FUNCTION func_name(_arg1 TEXT)
RETURNS INTEGER AS $$
    DECLARE
        var1 INTEGER;
        var2 TEXT;
    BEGIN
        -- Run a query (use semicolon at end). Variables will store values of first row
        SELECT id, name
        INTO   var1, var2
        FROM   table
        WHERE  address = $1;
        RETURN var1;
    END;
$$ language PLpgSQL; -- Determines type of function
```



# Week 4 Key Concepts

## PL/pgSQL loops

- **FOR** *i* **IN** *1..n* for fixed iteration, or **WHILE** for condition-based
- main use: iterating through query results row by row
- each row can be stored in multiple variables, a matching type, or *RECORD* (wildcard that adapts to whatever you're iterating)

```
... RETURNS TEXT AS $$
DECLARE
    _r RECORD;
    result TEXT = '';
BEGIN
    FOR _r IN -- Can also use FOR int_variable, text_variable IN
        SELECT id, name
        FROM   table
    LOOP
        -- Code inside runs for every row, record will have the values of the new row each time
        result = result || _r.id || _r.name || e'\n'; -- Record changed to a fitting tuple. _r.att
    END LOOP; -- Semicolon goes here, and not end of query.
    RETURN result;
END;
$$ language PLpgSQL; -- Determines type of function
```

```
DECLARE
    i INT := 1;
    total INT := 0;
BEGIN
    WHILE i <= 5 LOOP
        total := total + i;
        i := i + 1;
    END LOOP;
    RETURN total;
END;
```

# Week 4 Key Concepts

## PL/pgSQL returning tables

- You can use **RETURNS** SETOF *type* (type can be your own type), and use a variable *var* of that type to repeatedly **RETURN NEXT** *var* (make this variable the next tuple of my set of tuples).
- Or make your own table format by using **RETURNS TABLE**(*col1 TYPE*, *col2 TYPE*, ...) to return a table with given column names and types. The column names act as variables. Use **RETURN NEXT** to return a single row as the new row.

```
CREATE TYPE own_type AS (new_name TEXT, new_num INTEGER);
CREATE OR REPLACE FUNCTION func_name() RETURNS SETOF own_type
AS $$ -- SETOF can be a set of a single type, e.g. SETOF TEXT
DECLARE
    _r RECORD; result own_type;
BEGIN
    FOR _r IN
        SELECT id, name
        FROM table
    LOOP
        -- Fill in variable's fields
        result.new_name = 'New ' || _r.name;
        result.new_num = _r.id * 10;
        -- Return result variable as the next tuple of our
        -- function's result set
        RETURN NEXT result;
    END LOOP;
END;
$$ LANGUAGE PLpgSQL;
```

```
CREATE OR REPLACE FUNCTION func_name()
RETURNS TABLE(new_name TEXT, new_num INTEGER) AS $$
DECLARE
    _r RECORD;
BEGIN
    FOR _r IN
        SELECT id, name
        FROM table
    LOOP
        -- Directly address the column names as variables
        new_name = 'New' || _r.name;
        new_num = _r.id * 10;
        -- Return what is stored in each column as the
        -- next row of the table
        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE PLpgSQL;
```

# Week 4 Key Concepts

PL/pgSQL returning tables summary

SETOF vs **RETURNS TABLE**()

- SETOF type — use when returning rows that match an existing type/table structure
- **RETURNS TABLE**(...) — use when you need a custom column shape not tied to any existing type

# Week 4 Key Concepts

## PL/pgSQL IF Statements and Raising Notice/Exception

- IF statement syntax

```
IF cond THEN
    -- Logic;
ELSIF cond THEN
    -- Logic;
ELSE
    -- Logic;
END IF; -- Semicolon here, not anywhere else
```

```
DECLARE
    x INT := 10;
BEGIN
    IF x > 5 THEN
        RETURN 'big';
    ELSIF x = 5 THEN
        RETURN 'medium';
    ELSE
        RETURN 'small';
    END IF;
END;
```

- RAISE NOTICE 'msg' — print statement equivalent, useful for debugging
- RAISE EXCEPTION 'msg' — throws an error and halts execution (key for triggers)
- Use a % to represent a variable's value

```
RAISE NOTICE 'Value of var1 currently is: %', var1;
RAISE EXCEPTION 'AAAAAAAAAAAAAAAAAAAA WHAT DID YOU DO, % IS BADDDDDDDDDDDDDDDDDDDDD', var1;
```

# Tutorial Questions

Now let's try some more questions :D

- Q3
- Q4
- Q5
- Q7
- Q8
- Q14
- Q15
- Q16